1. **Aim of the practical:**
   To implement and evaluate an Artificial Neural Network (ANN) model using the MLPClassifier for classification

2. **Objective:**
   - To split the dataset into training and testing sets for model evaluation.
   - To build and train an Artificial Neural Network (ANN) using the MLPClassifier.
   - To predict the class labels for the test data using the trained ANN model.
   - To evaluate the model performance using a confusion matrix and visualize it with a heatmap.

3. **Algorithm/Flowchart:**

load_iris:

```
from sklearn.datasets import load_iris, make_moons, make_circles, make_blobs
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
X, y = load_iris(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

```python
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```python
model = MLPClassifier(hidden_layer_sizes=(10, 5), activation='relu', solver='adam', max_iter=500, random_state=42)
model.fit(X_train, y_train)
```

```
MLPClassifier
MLPClassifier(hidden_layer_sizes=(10, 5), max_iter=500, random_state=42)
```

```python
y_pred = model.predict(X_test)
```

```python
acc = accuracy_score(y_test, y_pred)
print(f"✅ Model Accuracy: {acc*100:.2f}%")
```

```
✅ Model Accuracy: 100.00%
```

```python
cm = confusion_matrix(y_test, y_pred)
```

```python
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm')
plt.title("Confusion Matrix Heatmap")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

Moons:

```python
from sklearn.datasets import load_iris, make_moons, make_circles, make_blobs
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = MLPClassifier(hidden_layer_sizes=(10, 5), activation='relu', solver='adam', max_iter=500, random_state=42)
model.fit(X_train, y_train)
```

```
MLPClassifier
MLPClassifier(hidden_layer_sizes=(10, 5), max_iter=500, random_state=42)
```

```
y_pred = model.predict(X_test)
```

```
acc = accuracy_score(y_test, y_pred)
print(f"✅ Model Accuracy: {acc*100:.2f}%")
```

```
✅ Model Accuracy: 97.60%
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm')
plt.title("Confusion Matrix Heatmap")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

circles:

```
from sklearn.datasets import load_iris, make_moons, make_circles, make_blobs
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
X, y = make_circles(n_samples=1000, noise=0.1, random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = MLPClassifier(hidden_layer_sizes=(10, 5), activation='relu', solver='adam', max_iter=500, random_state=42)
model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
  warnings.warn(
```

```
MLPClassifier
MLPClassifier(hidden_layer_sizes=(10, 5), max_iter=500, random_state=42)
```

```
y_pred = model.predict(X_test)
```

```
acc = accuracy_score(y_test, y_pred)
print(f"✅ Model Accuracy: {acc*100:.2f}%")
```

```
✅ Model Accuracy: 74.00%
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm')
plt.title("Confusion Matrix Heatmap")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

blobs:

```
from sklearn.datasets import load_iris, make_moons, make_circles, make_blobs
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
X, y = make_blobs(n_samples=1000, centers=2, cluster_std=2.0, random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = MLPClassifier(hidden_layer_sizes=(10, 5), activation='relu', solver='adam', max_iter=500, random_state=42)
model.fit(X_train, y_train)
```

```
MLPClassifier
MLPClassifier(hidden_layer_sizes=(10, 5), max_iter=500, random_state=42)
```

```
y_pred = model.predict(X_test)
```

```
acc = accuracy_score(y_test, y_pred)
print(f"✅ Model Accuracy: {acc*100:.2f}%")
```

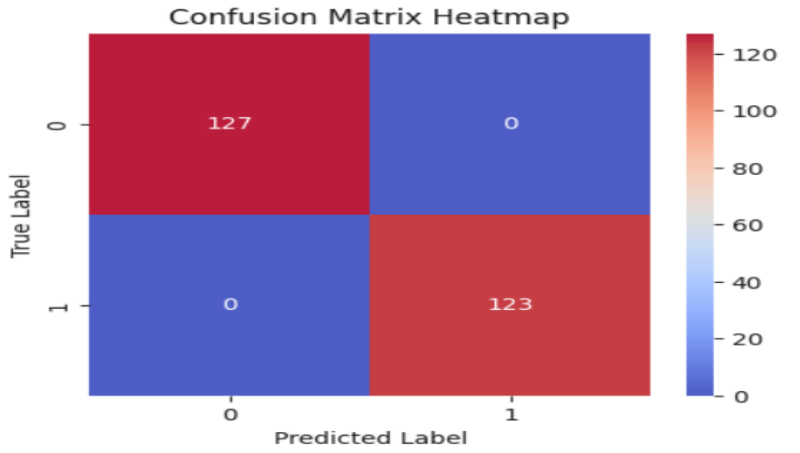
```
✅ Model Accuracy: 100.00%
```

```
cm = confusion_matrix(y_test, y_pred)
```

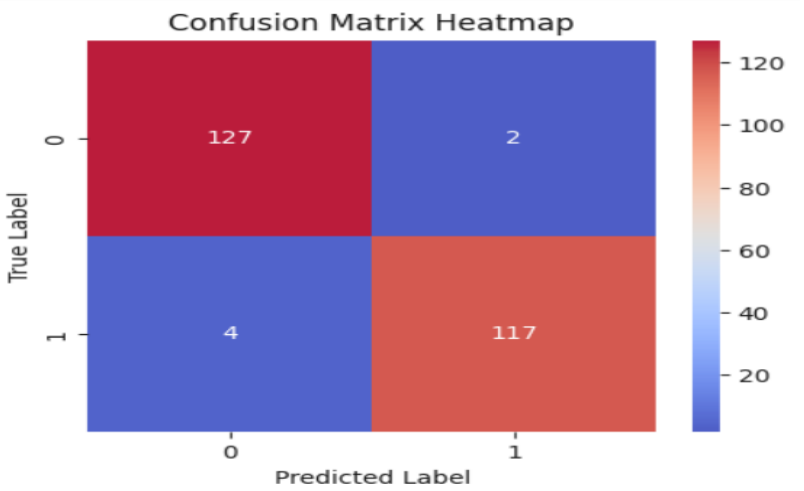
```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm')
plt.title("Confusion Matrix Heatmap")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

4. Output:

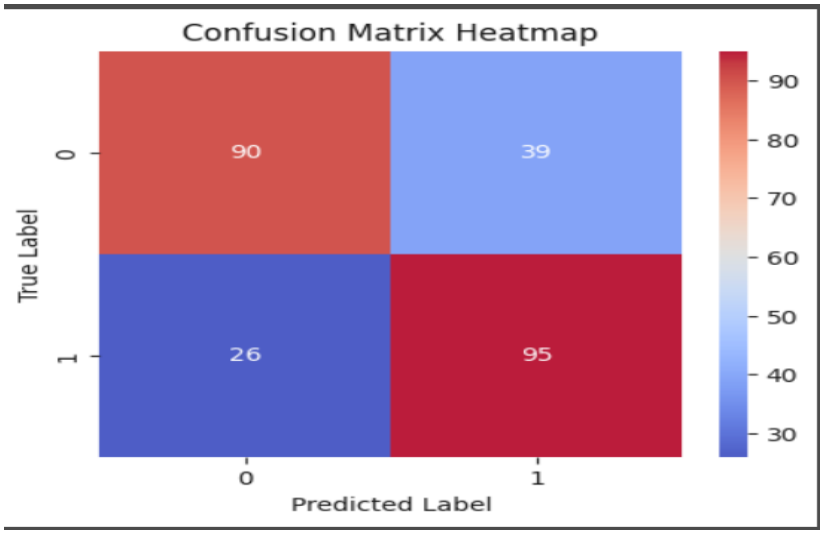
load_iris



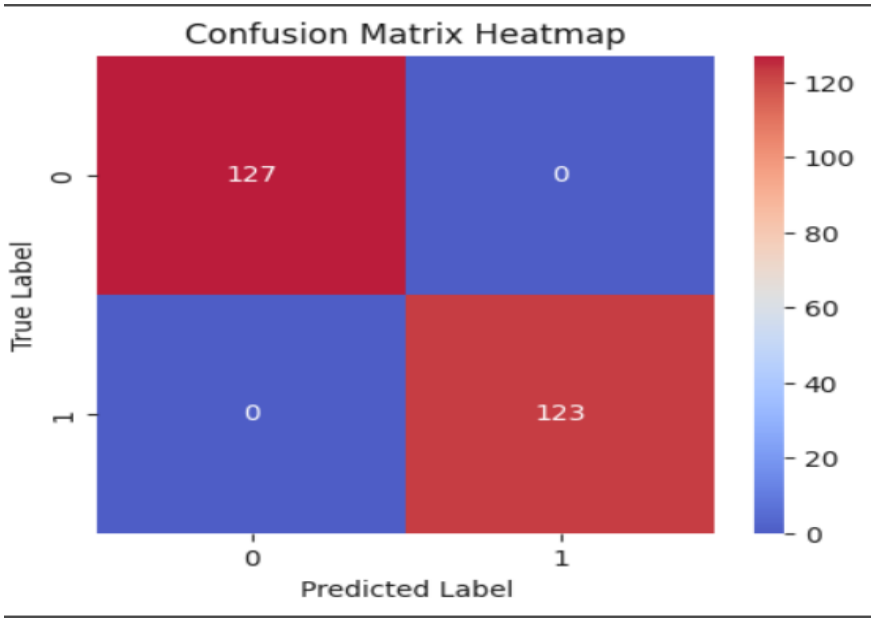
moons:



circles:



blobs:



5.Learning outcomes (What I have learnt):

I understand how to implement an Artificial Neural Network (ANN) using the MLPClassifier in Python.

I learnt how to split data into training and testing sets for model evaluation.

I gained hands-on experience in evaluating model accuracy using a confusion matrix.

I am able to visualize model performance using Seaborn's heatmap for better interpretation.